cardio_project

January 12, 2026

[1]:
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from dateutil import parser

from sklearn.model_selection import train_test_split, cross_val_score,␣
 ↪StratifiedKFold
from sklearn.pipeline import Pipeline
from sklearn.compose import ColumnTransformer
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import OneHotEncoder, StandardScaler
from sklearn.metrics import (
    confusion_matrix, roc_auc_score, roc_curve, accuracy_score,
    precision_recall_fscore_support, classification_report
)
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier, GradientBoostingClassifier

import seaborn as sns
```

[2]:
```
df = pd.read_csv("C:\\Users\\hp\\Desktop\\Project 3 - Healthcare - Predictive␣
 ↪Analytics\\Dataset\\cardio_data.csv")

print ("\Entire dataset:")
print(df)

print("\Column Information:")
df.info()
```

```
\Entire dataset:
             date    country     id  active    age  alco  ap_hi  ap_lo  \
0      03-05-2021  Indonesia      0       1  18393     0    110     80
1      05-08-2021   Malaysia      1       1  20228     0    140     90
2      13-11-2022  Indonesia      2       0  18857     0    130     70
3      31-10-2018  Singapore      3       1  17623     0    150    100
4      25-09-2020  Singapore      4       0  17474     0    100     60
…             …          …    …     …      …    …    …    …    …
69995  03-04-2018  Singapore  99993       1  19240     0    120     80
```

```
69996  12-01-2022   Malaysia  99995       1  22601     0    140     90
69997  25-08-2022   Malaysia  99996       0  19066     1    180     90
69998  13-07-2020  Singapore  99998       0  22431     0    135     80
69999  15-01-2018  Singapore  99999       1  20540     0    120     80

       cholesterol  gender  gluc  height  occupation  smoke  weight  disease
0                1       2     1     168   Architect      0    62.0        0
1                3       1     1     156  Accountant      0    85.0        1
2                3       1     1     165        Chef      0    64.0        1
3                1       2     1     169      Lawyer      0    82.0        1
4                1       1     1     156   Architect      0    56.0        0
…              …     …   …     …       …         …    …      …       …
69995            1       2     1     168      Doctor      1    76.0        0
69996            2       1     2     158  Accountant      0   126.0        1
69997            3       2     1     183  Accountant      0   105.0        1
69998            1       1     2     163  Accountant      0    72.0        1
69999            2       1     1     170     Teacher      0    72.0        0

[70000 rows x 16 columns]
\Column Information:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 70000 entries, 0 to 69999
Data columns (total 16 columns):
 #   Column       Non-Null Count  Dtype
---  ------       --------------  -----
 0   date         70000 non-null  object
 1   country      70000 non-null  object
 2   id           70000 non-null  int64
 3   active       70000 non-null  int64
 4   age          70000 non-null  int64
 5   alco         70000 non-null  int64
 6   ap_hi        70000 non-null  int64
 7   ap_lo        70000 non-null  int64
 8   cholesterol  70000 non-null  int64
 9   gender       70000 non-null  int64
 10  gluc         70000 non-null  int64
 11  height       70000 non-null  int64
 12  occupation   70000 non-null  object
 13  smoke        70000 non-null  int64
 14  weight       70000 non-null  float64
 15  disease      70000 non-null  int64
dtypes: float64(1), int64(12), object(3)
memory usage: 8.5+ MB
```

[3]:
```python
df = df.rename(columns={
    "gluc": "glucose",
})
```

[4]:
```
def parse_mixed_date(x):
    try:
        return parser.parse(str(x), dayfirst=True)
    except Exception:
        return np.nan

df["date_parsed"] = df["date"].apply(parse_mixed_date)
df["year"] = df["date_parsed"].dt.year
df["month"] = df["date_parsed"].dt.month
df["day"] = df["date_parsed"].dt.day
df["dayofweek"] = df["date_parsed"].dt.dayofweek
```

[5]:
```
df["age_years"] = df["age"] / 365.25
```

[6]:
```
def clean_bp(row):
    hi, lo = row["ap_hi"], row["ap_lo"]
    if pd.notna(lo) and lo > 300:
        lo = lo / 10.0
    if pd.notna(hi) and hi > 300:
        hi = hi / 10.0
    if pd.notna(hi) and pd.notna(lo) and lo > hi:
        hi, lo = lo, hi
    return pd.Series({"ap_hi": hi, "ap_lo": lo})

df[["ap_hi", "ap_lo"]] = df.apply(clean_bp, axis=1)
df["ap_hi"] = df["ap_hi"].clip(lower=70, upper=250)
df["ap_lo"] = df["ap_lo"].clip(lower=40, upper=150)
```

[7]:
```
df["height_m"] = df["height"] / 100.0
df["BMI"] = df["weight"] / (df["height_m"] ** 2)
df["BMI"] = df["BMI"].clip(lower=10, upper=70)

df["pulse_pressure"] = df["ap_hi"] - df["ap_lo"]
df["MAP"] = df["ap_lo"] + df["pulse_pressure"] / 3.0
```

[8]:
```
df["hypertension_stage1"] = (
    ((df["ap_hi"] >= 130) & (df["ap_hi"] <= 139)) |
    ((df["ap_lo"] >= 80) & (df["ap_lo"] <= 89))
).astype(int)
df["hypertension_stage2"] = ((df["ap_hi"] >= 140) | (df["ap_lo"] >= 90)).
 ↪astype(int)
df["hypertension_any"] = ((df["ap_hi"] >= 130) | (df["ap_lo"] >= 80)).
 ↪astype(int)
df["pp_high"] = (df["pulse_pressure"] >= 60).astype(int)
```

[9]:
```
df["cholesterol"] = pd.to_numeric(df["cholesterol"], errors="coerce")
```

```
for col in ["alco", "smoke", "active"]:
    df[col] = pd.to_numeric(df[col], errors="coerce").clip(0, 1)
```

[10]:
```
df["gender"] = pd.to_numeric(df["gender"], errors="coerce")
```

[11]:
```
df["BMI_class"] = pd.cut(
    df["BMI"], bins=[0, 18.5, 25, 30, 70],
    labels=["underweight", "normal", "overweight", "obese"]
)
df["age_bin"] = pd.cut(
    df["age_years"], bins=[0, 30, 40, 50, 60, 70, 120],
    labels=["<30", "30-39", "40-49", "50-59", "60-69", "70+"]
)
```

[12]:
```
df = df.drop(columns=["id", "date", "date_parsed", "height_m"])
```

[13]:
```
df = df.dropna(subset=["disease"])
df["disease"] = pd.to_numeric(df["disease"], errors="coerce").astype(int)

numeric_features = [
    "age_years", "ap_hi", "ap_lo", "pulse_pressure", "MAP",
    "BMI", "cholesterol", "gluc", "year", "month", "day", "dayofweek"
]
numeric_features = [f for f in numeric_features if f in df.columns]

binary_features = [
    "alco", "smoke", "active", "hypertension_stage1",
    "hypertension_stage2", "hypertension_any", "pp_high"
]
binary_features = [f for f in binary_features if f in df.columns]

categorical_features = ["gender", "country", "occupation", "BMI_class",␣
 ↪"age_bin"]
categorical_features = [f for f in categorical_features if f in df.columns]
```

[14]:
```
for col in numeric_features:
    df[col] = pd.to_numeric(df[col], errors="coerce")
for col in binary_features:
    df[col] = df[col].astype(int)
for col in categorical_features:
    df[col] = df[col].astype("category")

X = df[numeric_features + binary_features + categorical_features]
y = df["disease"].values

print ("\nEntire dataset:")
print(df)
```

```
print("\nColumn Information:")
df.info()
```

```
Entire dataset:
         country  active    age  alco  ap_hi  ap_lo  cholesterol gender  \
0      Indonesia       1  18393     0  110.0   80.0            1      2
1       Malaysia       1  20228     0  140.0   90.0            3      1
2      Indonesia       0  18857     0  130.0   70.0            3      1
3      Singapore       1  17623     0  150.0  100.0            1      2
4      Singapore       0  17474     0  100.0   60.0            1      1
…            …     …    …     …    …     …    …          …    …
69995  Singapore       1  19240     0  120.0   80.0            1      2
69996   Malaysia       1  22601     0  140.0   90.0            2      1
69997   Malaysia       0  19066     1  180.0   90.0            3      2
69998  Singapore       0  22431     0  135.0   80.0            1      1
69999  Singapore       1  20540     0  120.0   80.0            2      1

       glucose  height  …  age_years        BMI  pulse_pressure         MAP  \
0            1     168  …  50.357290  21.967120            30.0   90.000000
1            1     156  …  55.381246  34.927679            50.0  106.666667
2            1     165  …  51.627652  23.507805            60.0   90.000000
3            1     169  …  48.249144  28.710479            50.0  116.666667
4            1     156  …  47.841205  23.011177            40.0   73.333333
…          …     …  …          …         …              …          …
69995        1     168  …  52.676249  26.927438            40.0   93.333333
69996        2     158  …  61.878166  50.472681            50.0  106.666667
69997        1     183  …  52.199863  31.353579            90.0  120.000000
69998        2     163  …  61.412731  27.099251            55.0   98.333333
69999        1     170  …  56.235455  24.913495            40.0   93.333333

       hypertension_stage1  hypertension_stage2  hypertension_any  pp_high  \
0                        1                    0                 1        0
1                        0                    1                 1        0
2                        1                    0                 1        1
3                        0                    1                 1        0
4                        0                    0                 0        0
…                      …                    …                 …       …
69995                    1                    0                 1        0
69996                    0                    1                 1        0
69997                    0                    1                 1        1
69998                    1                    0                 1        0
69999                    1                    0                 1        0

        BMI_class  age_bin
0          normal    50-59
1           obese    50-59
```

```
2          normal    50-59
3      overweight    40-49
4          normal    40-49
…           …      …
69995  overweight    50-59
69996       obese    60-69
69997       obese    50-59
69998  overweight    60-69
69999      normal    50-59

[70000 rows x 28 columns]

Column Information:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 70000 entries, 0 to 69999
Data columns (total 28 columns):
 #   Column               Non-Null Count  Dtype
---  ------               --------------  -----
 0   country              70000 non-null  category
 1   active               70000 non-null  int64
 2   age                  70000 non-null  int64
 3   alco                 70000 non-null  int64
 4   ap_hi                70000 non-null  float64
 5   ap_lo                70000 non-null  float64
 6   cholesterol          70000 non-null  int64
 7   gender               70000 non-null  category
 8   glucose              70000 non-null  int64
 9   height               70000 non-null  int64
 10  occupation           70000 non-null  category
 11  smoke                70000 non-null  int64
 12  weight               70000 non-null  float64
 13  disease              70000 non-null  int64
 14  year                 70000 non-null  int32
 15  month                70000 non-null  int32
 16  day                  70000 non-null  int32
 17  dayofweek            70000 non-null  int32
 18  age_years            70000 non-null  float64
 19  BMI                  70000 non-null  float64
 20  pulse_pressure       70000 non-null  float64
 21  MAP                  70000 non-null  float64
 22  hypertension_stage1  70000 non-null  int64
 23  hypertension_stage2  70000 non-null  int64
 24  hypertension_any     70000 non-null  int64
 25  pp_high              70000 non-null  int64
 26  BMI_class            70000 non-null  category
 27  age_bin              70000 non-null  category
dtypes: category(5), float64(7), int32(4), int64(12)
memory usage: 11.6 MB
```

[15]:
```
df.to_csv(r"C:\Users\hp\Desktop\cardio_data_cleaned3.csv", index=False)
```

[16]:
```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)
```

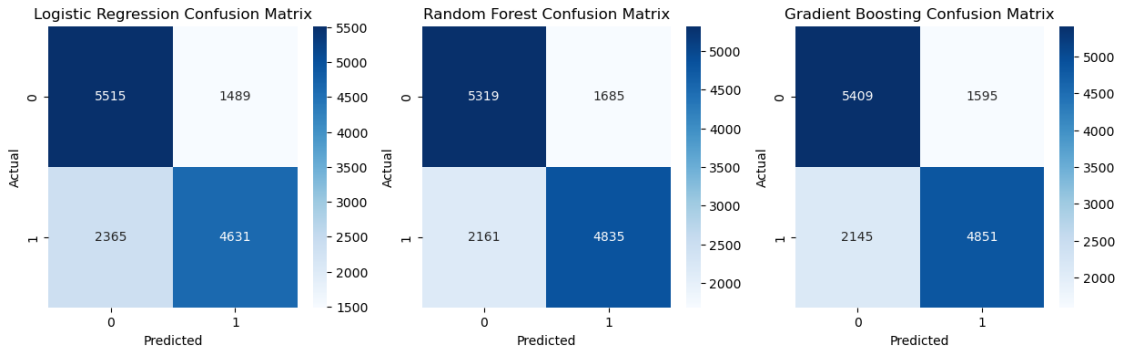
[17]:
```
num_transformer = Pipeline(steps=[
    ("imputer", SimpleImputer(strategy="median")),
    ("scaler", StandardScaler())
])

cat_transformer = Pipeline(steps=[
    ("imputer", SimpleImputer(strategy="most_frequent")),
    ("onehot", OneHotEncoder(handle_unknown="ignore", sparse_output=False))
])

bin_transformer = Pipeline(steps=[
    ("imputer", SimpleImputer(strategy="most_frequent"))
])

preprocessor = ColumnTransformer(
    transformers=[
        ("num", num_transformer, numeric_features),
        ("cat", cat_transformer, categorical_features),
        ("bin", bin_transformer, binary_features),
    ],
    remainder="drop"
)
```

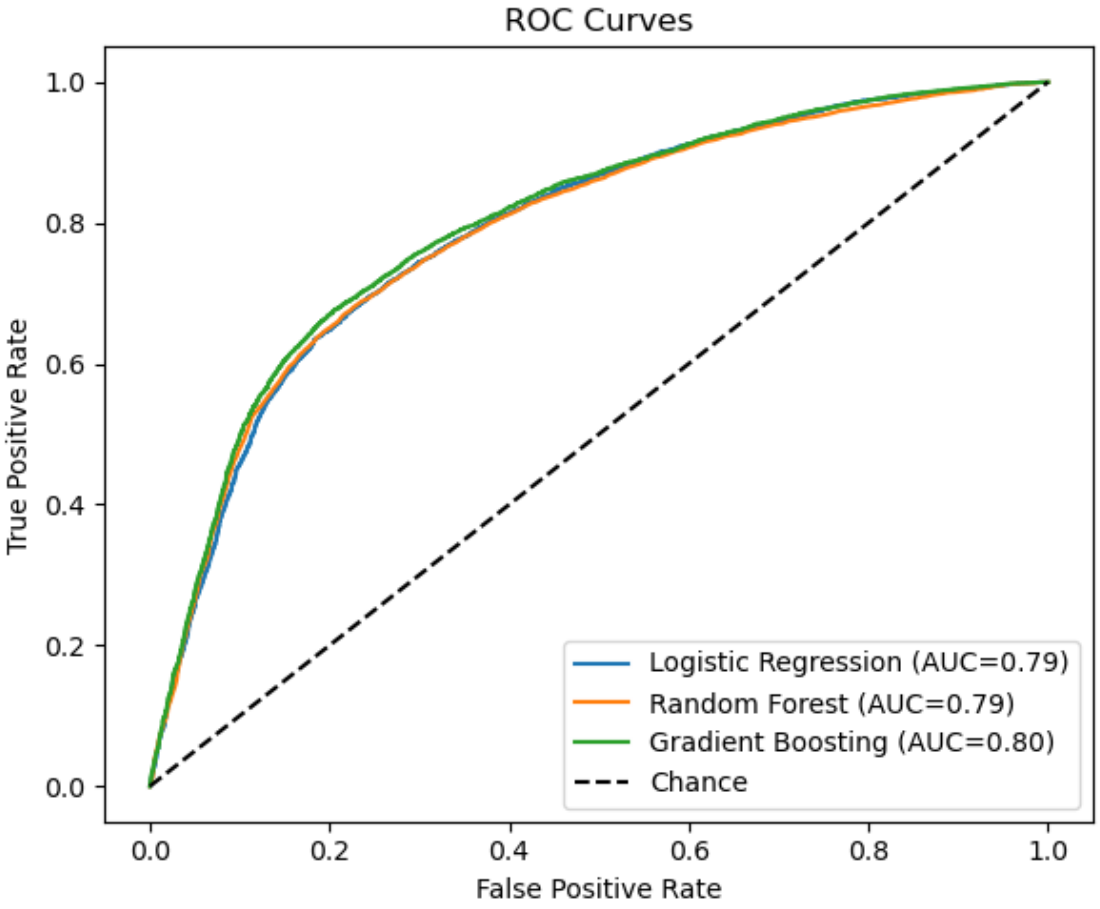
[18]:
```
models = {
    "Logistic Regression": LogisticRegression(max_iter=1000,␣
 ↪class_weight="balanced"),
    "Random Forest": RandomForestClassifier(
        n_estimators=300, class_weight="balanced_subsample", random_state=42
    ),
    "Gradient Boosting": GradientBoostingClassifier(random_state=42)
}
```

[19]:
```
cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
results = []

fig_cm, axes_cm = plt.subplots(1, 3, figsize=(15, 4))
fig_roc, ax_roc = plt.subplots(figsize=(6, 5))

for i, (name, model) in enumerate(models.items()):
    pipe = Pipeline([("preprocess", preprocessor), ("model", model)])
    pipe.fit(X_train, y_train)
```

```
    y_pred = pipe.predict(X_test)
    y_proba = pipe.predict_proba(X_test)[:, 1] if hasattr(model,␣
↪"predict_proba") else None

    acc = accuracy_score(y_test, y_pred)
    precision, recall, f1, _ = precision_recall_fscore_support(y_test, y_pred,␣
↪average="binary")
    roc_auc = roc_auc_score(y_test, y_proba) if y_proba is not None else np.nan
    cv_scores = cross_val_score(pipe, X, y, cv=cv, scoring="accuracy")

    results.append({
        "model": name,
        "accuracy": acc,
        "precision": precision,
        "recall": recall,
        "f1": f1,
        "roc_auc": roc_auc,
        "cv_mean_acc": cv_scores.mean(),
        "cv_std_acc": cv_scores.std()
    })

    cm = confusion_matrix(y_test, y_pred)
    sns.heatmap(cm, annot=True, fmt="d", cmap="Blues", ax=axes_cm[i])
    axes_cm[i].set_title(f"{name} Confusion Matrix")
    axes_cm[i].set_xlabel("Predicted")
    axes_cm[i].set_ylabel("Actual")

    if y_proba is not None:
        fpr, tpr, _ = roc_curve(y_test, y_proba)
        ax_roc.plot(fpr, tpr, label=f"{name} (AUC={roc_auc:.2f})")

ax_roc.plot([0, 1], [0, 1], "k--", label="Chance")
ax_roc.set_title("ROC Curves")
ax_roc.set_xlabel("False Positive Rate")
ax_roc.set_ylabel("True Positive Rate")
ax_roc.legend(loc="lower right")

plt.tight_layout()
print("Confusion matrices and ROC curves plotted successfully")
plt.show()
```

```
Confusion matrices and ROC curves plotted successfully
```





[20]:
```
def get_feature_names(preprocessor, numeric_features, categorical_features,␣
↪binary_features):
    num_names = numeric_features
    cat_pipeline = preprocessor.named_transformers_["cat"]
    ohe = cat_pipeline.named_steps["onehot"]
```

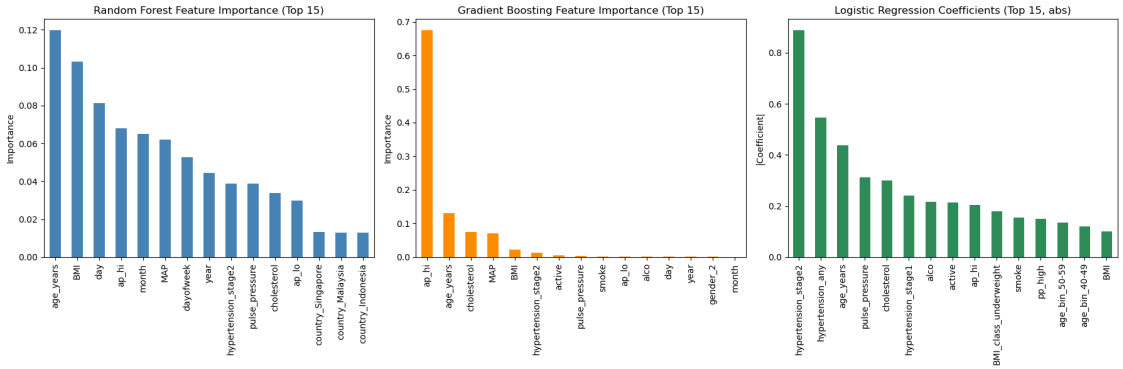
```
    cat_names = ohe.get_feature_names_out(categorical_features).tolist()
    bin_names = binary_features
    return num_names + cat_names + bin_names

feature_names = get_feature_names(preprocessor, numeric_features,␣
 ↪categorical_features, binary_features)
```

[21]:
```
rf_pipe = Pipeline([("preprocess", preprocessor), ("model", models["Random␣
 ↪Forest"])])
rf_pipe.fit(X_train, y_train)
rf_model = rf_pipe.named_steps["model"]
rf_importances = pd.Series(rf_model.feature_importances_, index=feature_names).
 ↪sort_values(ascending=False)

gb_pipe = Pipeline([("preprocess", preprocessor), ("model", models["Gradient␣
 ↪Boosting"])])
gb_pipe.fit(X_train, y_train)
gb_model = gb_pipe.named_steps["model"]
gb_importances = pd.Series(gb_model.feature_importances_, index=feature_names).
 ↪sort_values(ascending=False)

log_pipe = Pipeline([("preprocess", preprocessor), ("model", models["Logistic␣
 ↪Regression"])])
log_pipe.fit(X_train, y_train)
log_model = log_pipe.named_steps["model"]
log_coefs = pd.Series(np.abs(log_model.coef_[0]), index=feature_names).
 ↪sort_values(ascending=False)

print("Random Forest Top Features:\n", rf_importances.head(10))
print("\nGradient Boosting Top Features:\n", gb_importances.head(10))
print("\nLogistic Regression Top Coefficients:\n", log_coefs.head(10))
```

```
Random Forest Top Features:
 age_years              0.119669
BMI                     0.103302
day                     0.081302
ap_hi                   0.068123
month                   0.064839
MAP                     0.062043
dayofweek               0.052725
year                    0.044293
hypertension_stage2     0.038895
pulse_pressure          0.038727
dtype: float64

Gradient Boosting Top Features:
 ap_hi                  0.674268
```

```
age_years              0.130153
cholesterol            0.074258
MAP                    0.070763
BMI                    0.022103
hypertension_stage2    0.012120
active                 0.004474
pulse_pressure         0.002808
smoke                  0.001726
ap_lo                  0.001151
dtype: float64

Logistic Regression Top Coefficients:
 hypertension_stage2      0.887698
hypertension_any         0.545447
age_years                0.438733
pulse_pressure           0.312964
cholesterol              0.299654
hypertension_stage1      0.241778
alco                     0.217031
active                   0.214831
ap_hi                    0.203726
BMI_class_underweight    0.178556
dtype: float64
```

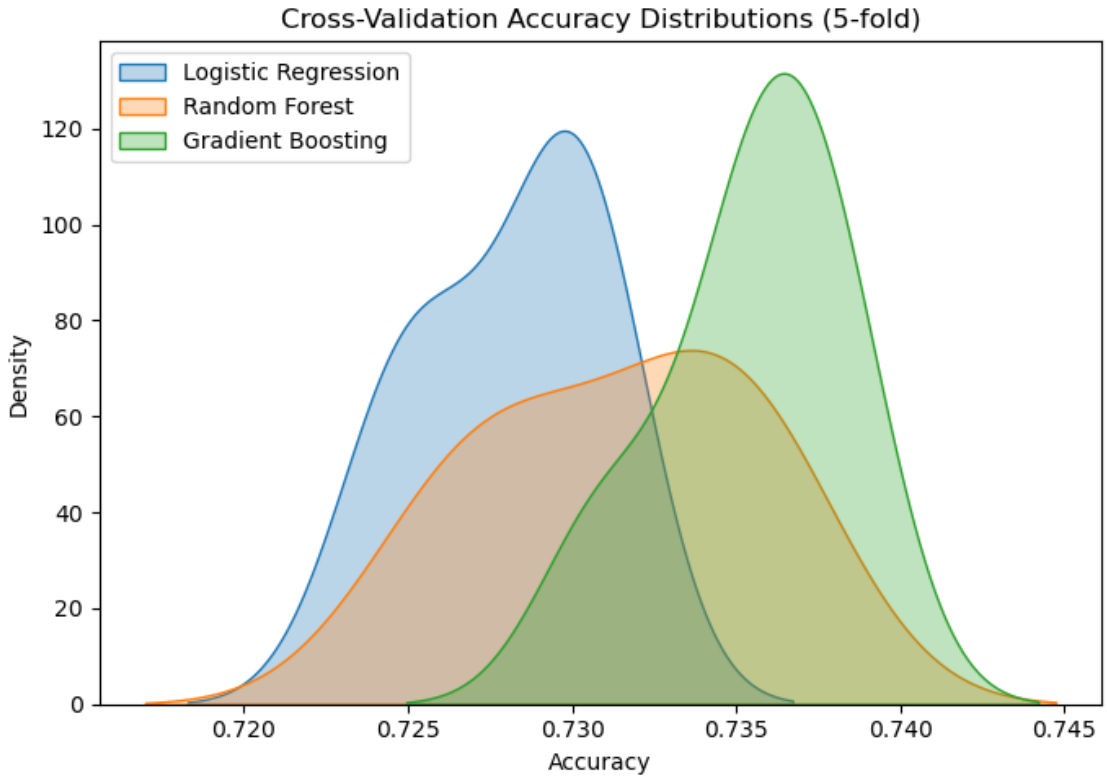
[22]:
```
fig_imp, ax_imp = plt.subplots(1, 3, figsize=(18, 6))
rf_importances.head(15).plot(kind="bar", ax=ax_imp[0], color="steelblue")
ax_imp[0].set_title("Random Forest Feature Importance (Top 15)")
ax_imp[0].set_ylabel("Importance")

gb_importances.head(15).plot(kind="bar", ax=ax_imp[1], color="darkorange")
ax_imp[1].set_title("Gradient Boosting Feature Importance (Top 15)")
ax_imp[1].set_ylabel("Importance")

log_coefs.head(15).plot(kind="bar", ax=ax_imp[2], color="seagreen")
ax_imp[2].set_title("Logistic Regression Coefficients (Top 15, abs)")
ax_imp[2].set_ylabel("|Coefficient|")

plt.tight_layout()
plt.show()
```



[23]:
```
comparison_df = pd.DataFrame({
    "Random Forest": rf_importances,
    "Gradient Boosting": gb_importances,
    "Logistic Regression": log_coefs
})

print(comparison_df.head(10))  # top features across models
```

	Random Forest	Gradient Boosting	Logistic Regression
BMI	0.103302	0.022103	0.100363
BMI_class_normal	0.007919	0.000000	0.064251
BMI_class_obese	0.007331	0.000000	0.062592
BMI_class_overweight	0.008176	0.000022	0.030240
BMI_class_underweight	0.000953	0.000000	0.178556
MAP	0.062043	0.070763	0.089472
active	0.012769	0.004474	0.214831
age_bin_30-39	0.002150	0.000000	0.044547
age_bin_40-49	0.007955	0.000000	0.120585
age_bin_50-59	0.006919	0.000332	0.133849

[24]:
```
fig_cv, ax_cv = plt.subplots(figsize=(7, 5))
for name, model in models.items():
    pipe = Pipeline([("preprocess", preprocessor), ("model", model)])
    cv_scores = cross_val_score(pipe, X, y, cv=cv, scoring="accuracy")
    sns.kdeplot(cv_scores, label=name, ax=ax_cv, fill=True, alpha=0.3)
ax_cv.set_title("Cross-Validation Accuracy Distributions (5-fold)")
ax_cv.set_xlabel("Accuracy")
ax_cv.legend()
plt.tight_layout()
plt.show()
```

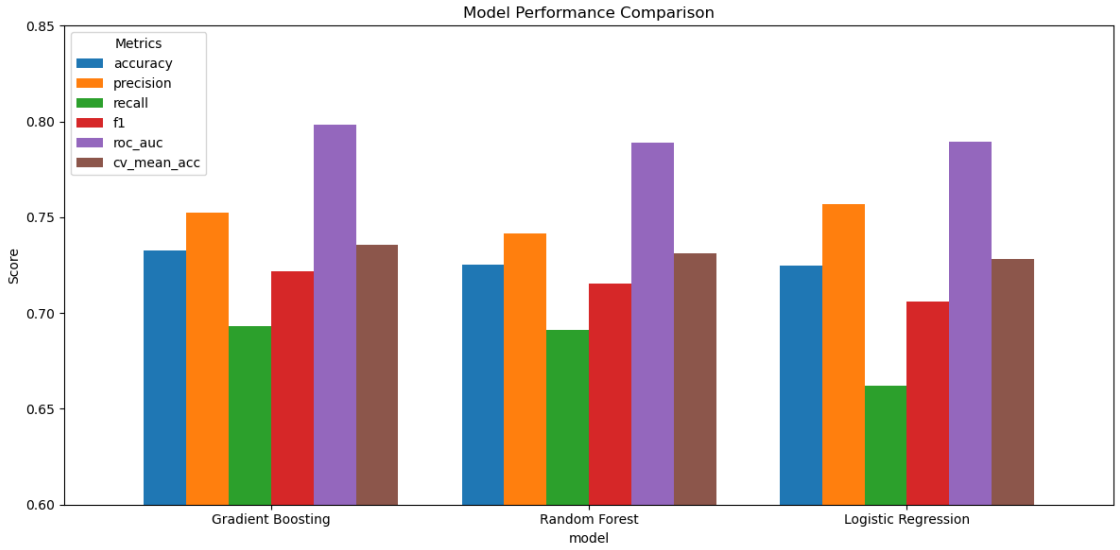


[25]:
```
results_df = pd.DataFrame(results).sort_values(by="accuracy", ascending=False)
print("\nModel Performance Summary:")
print(results_df.round(4))
```

```
Model Performance Summary:
                 model  accuracy  precision  recall      f1  roc_auc  \
2    Gradient Boosting    0.7329     0.7526  0.6934  0.7218   0.7981
1        Random Forest    0.7253     0.7416  0.6911  0.7154   0.7888
0  Logistic Regression    0.7247     0.7567  0.6619  0.7062   0.7897

   cv_mean_acc  cv_std_acc
2       0.7354      0.0025
1       0.7314      0.0037
0       0.7281      0.0025
```

[26]:
```
metrics = ["accuracy", "precision", "recall", "f1", "roc_auc", "cv_mean_acc"]

# Plot grouped bar chart
ax = results_df.set_index("model")[metrics].plot(
    kind="bar",
    figsize=(12, 6),
```

```
    width=0.8
)

# Formatting
plt.title("Model Performance Comparison")
plt.ylabel("Score")
plt.ylim(0.6, 0.85)  # adjust based on your metric ranges
plt.legend(title="Metrics")
plt.xticks(rotation=0)
plt.tight_layout()
plt.show()
```

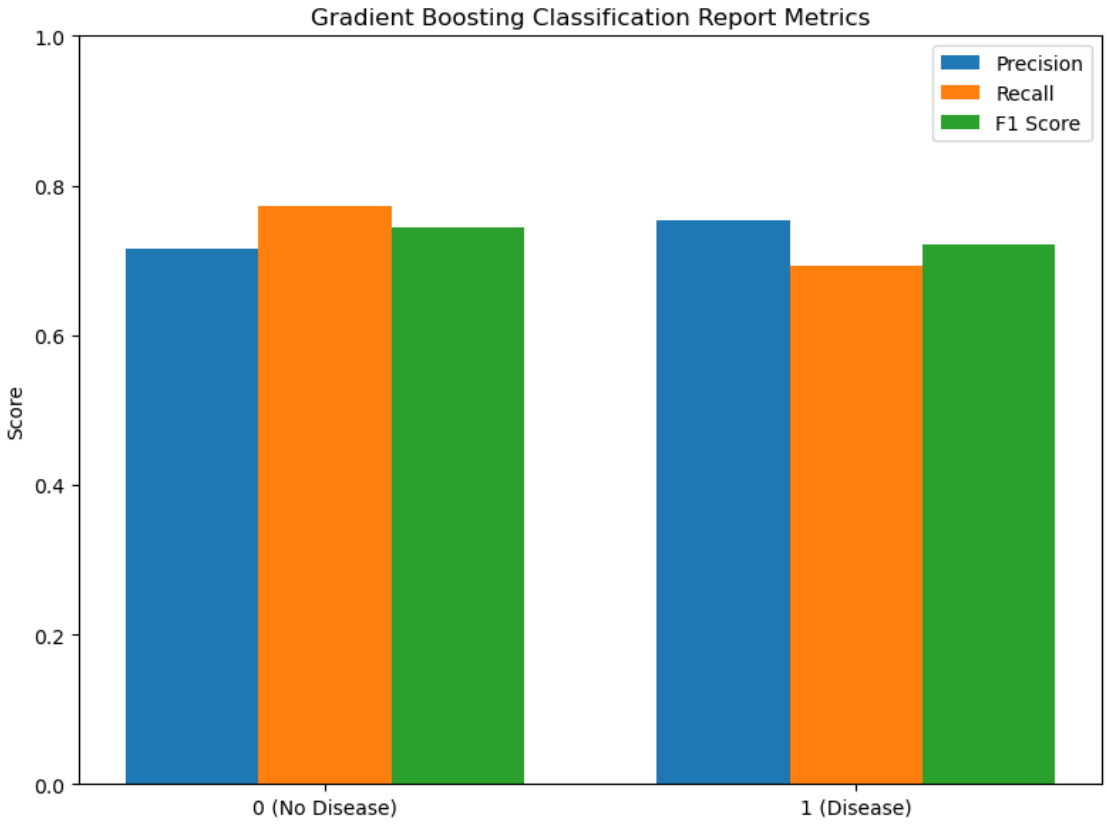


[27]:
```
best_name = results_df.iloc[0]["model"]
best_pipe = Pipeline([("preprocess", preprocessor), ("model",␣
 ↪models[best_name])])
best_pipe.fit(X_train, y_train)
y_pred_best = best_pipe.predict(X_test)
print(f"\nDetailed Classification Report - {best_name}:")
print(classification_report(y_test, y_pred_best))
```

```
Detailed Classification Report - Gradient Boosting:
              precision    recall  f1-score   support

           0       0.72      0.77      0.74      7004
           1       0.75      0.69      0.72      6996

    accuracy                           0.73     14000
   macro avg       0.73      0.73      0.73     14000
```

```
weighted avg       0.73      0.73      0.73      14000
```

[30]:
```python
report = classification_report(y_test, y_pred_best, output_dict=True)

# Extract metrics for classes 0 and 1
classes = ['0 (No Disease)', '1 (Disease)']
precision = [report['0']['precision'], report['1']['precision']]
recall = [report['0']['recall'], report['1']['recall']]
f1 = [report['0']['f1-score'], report['1']['f1-score']]

x = np.arange(len(classes))
width = 0.25

fig, ax = plt.subplots(figsize=(8,6))
ax.bar(x - width, precision, width, label='Precision')
ax.bar(x, recall, width, label='Recall')
ax.bar(x + width, f1, width, label='F1 Score')

ax.set_xticks(x)
ax.set_xticklabels(classes)
ax.set_ylim(0,1)
ax.set_ylabel("Score")
ax.set_title("Gradient Boosting Classification Report Metrics")
ax.legend()

plt.tight_layout()
plt.show()
```



[31]:
```
import joblib
```

[32]:
```
best_model_name = "Gradient Boosting"
best_pipe = Pipeline([("preprocess", preprocessor), ("model",␣
 ↪models[best_model_name])])
best_pipe.fit(X_train, y_train)
```

[32]:
```
Pipeline(steps=[('preprocess',
                 ColumnTransformer(transformers=[('num',
                                                  Pipeline(steps=[('imputer',
SimpleImputer(strategy='median')),
                                                                  ('scaler',
StandardScaler())]),
                                                  ['age_years', 'ap_hi',
                                                   'ap_lo', 'pulse_pressure',
                                                   'MAP', 'BMI', 'cholesterol',
                                                   'year', 'month', 'day',
                                                   'dayofweek']),
                                                 ('cat',
                                                  Pipeline(steps=[('imputer',
SimpleImputer(strategy='most_frequent')),
```

```
                                                                  ('onehot',
OneHotEncoder(handle_unknown='ignore',
 sparse_output=False))]),
                                                   ['gender', 'country',
                                                    'occupation', 'BMI_class',
                                                    'age_bin']),
                                                  ('bin',
                                                   Pipeline(steps=[('imputer',
SimpleImputer(strategy='most_frequent'))]),
                                                   ['alco', 'smoke', 'active',
                                                    'hypertension_stage1',
                                                    'hypertension_stage2',
                                                    'hypertension_any',
                                                    'pp_high'])])),
                ('model', GradientBoostingClassifier(random_state=42))])
```

[33]:
```
joblib.dump(best_pipe, "best_gradient_boosting_pipeline.ppt")
print("  Best model saved as 'best_gradient_boosting_pipeline.ppt'")
```

```
 Best model saved as 'best_gradient_boosting_pipeline.ppt'
```

[34]:
```
def predict(patient_row):
    """
    Predict heart disease for a single patient record.
    patient_row: dict with keys matching feature names (age_years, ap_hi,␣
 ↪ap_lo, BMI, cholesterol, gluc, etc.)
    Returns: prediction (0 = no disease, 1 = disease) and probability """

    patient_df = pd.DataFrame([patient_row])
    pred = best_pipe.predict(patient_df)[0]
    proba = best_pipe.predict_proba(patient_df)[0, 1]
    return pred, proba
```

[35]:
```
example_patient = {
    "age_years": 55,
    "ap_hi": 145,
    "ap_lo": 95,
    "pulse_pressure": 50,
    "MAP": 112,
    "BMI": 29,
    "cholesterol": 2,
    "gluc": 1,
    "year": 2025,
    "month": 1,
    "day": 10,
    "dayofweek": 6,
    "alco": 0,
    "smoke": 0,
```

```
    "active": 1,
    "hypertension_stage1": 0,
    "hypertension_stage2": 1,
    "hypertension_any": 1,
    "pp_high": 0,
    "gender": 1,
    "country": "Nigeria",
    "occupation": "Engineer",
    "BMI_class": "overweight",
    "age_bin": "50-59"
}

pred, proba = predict(example_patient)
print(f"Prediction: {'Disease' if pred==1 else 'No Disease'}␣
 ↪(probability={proba:.2f})")
```

```
Prediction: Disease (probability=0.71)
```

[ ]: